46. Битовые флаги и битовые маски

На этом уроке мы рассмотрим битовые флаги и битовые маски в языке C++.

Примечание: Для некоторых читателей этот материал может показаться немного сложным. Если вы застряли или что-то не понятно — пропустите его. Вы можете смело к нему вернуться, когда будете достаточно детальны. Он не стоит таких же сил, как для прохода в изучении языка C++, как другие уроки, и изложен здесь в большей мере для общего развития.

Оглавление:

1. Битовые флаги
2. Почему битовые флаги полезны?
3. Введение в std::bitset
4. Битовые маски
5. Пример с RGBA
6. Заключение
7. Тест
8. Ответы

Битовые флаги

Используя целый тип данных значения **логического типа данных**, вы занимаете только 1 бит, а остальные 7 из 8 — не используются. Хотя в целом это нормально, в особых, редкоубиваемых случаях, требующих с экономии постоянной памяти, может быть полезно использовать 8 значений того же типа в 1 байт, сохранив при этом место или память и увеличив, таким образом, производительность. Это отдельные биты и называются **битовыми флагами**. Поскольку доступ к этим битам напрямую отсутствует, то для операций с ними используются **побитовые операторы**.

Примечание: На этом уроке мы будем использовать значения из шестнадцатеричной системы счисления.

Например:

```
const unsigned char mask0 = 0x01; // 00000001 (0-й бит)
const unsigned char mask1 = 0x02; // 00000010 (1-й бит)
const unsigned char mask2 = 0x04; // 00000100 (2-й бит)
const unsigned char mask3 = 0x08; // 00000100 (3-й бит)
const unsigned char mask4 = 0x10; // 00000100 (4-й бит)
const unsigned char mask5 = 0x20; // 00000100 (5-й бит)
const unsigned char mask6 = 0x40; // 00000100 (6-й бит)
const unsigned char mask7 = 0x80; // 00000100 (7-й бит)

// Определим переменную для хранения наших 8 значений
unsigned char flags = 0x00;
```

Чтобы узнать битовое состояние, используется побитовое И:

```
bool bIsFlag = flags & mask0;
```

Чтобы включить бит, используется побитовое ИЛИ:

```
flags |= mask0; // включим первый бит
flags |= mask1; // включим второй бит
```

Чтобы выключить бит, используется побитовое И с инвертированным литералом:

```
flags &= ~mask0; // выключим первый бит
flags &= ~mask1; // выключим второй бит
```

А чтобы проверить между состоянием бит, используется побитовое исключающее ИЛИ (XOR):

```
flags ^= mask0; // переключим первый бит
```

В качестве примера возьмем библиотеку 3D графики OpenGL, в которой некоторые функции принимают один или несколько битовых флагов в качестве параметров:

```
GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT // определяется следующим образом (в gl.h):
#define GL_COLOR_BUFFER_BIT 0x00004000
#define GL_DEPTH_BUFFER_BIT 0x00000100
```

Вот небольшой пример:

```
// Libs:
// Flags:
int main()
{
    // Определим переменную для хранения наших флагов
    const unsigned char isHungry = 0x01; // 00000001 (0-й бит)
    const unsigned char isSad = 0x02; // 00000010 (1-й бит)
    const unsigned char isMad = 0x04; // 00000100 (2-й бит)
    const unsigned char isHappy = 0x08; // 00000100 (3-й бит)
    const unsigned char isSleepy = 0x10; // 00000100 (4-й бит)
    const unsigned char isNaked = 0x20; // 00000100 (5-й бит)
    const unsigned char isHead = 0x40; // 00000100 (6-й бит)
    const unsigned char isFeet = 0x80; // 00000100 (7-й бит)

    unsigned char me = 0; // определим переменную от имени
    me |= isHungry | isHappy; // я голодный и счастливый
    me |= isLaughing; // я также смеюсь

    std::cout << "I am happy?" << (bool(me & isHappy)) << '\n';
    std::cout << "I am laughing?" << (bool(me & isLaughing)) << '\n';

    return 0;
}
```

Почему битовые флаги полезны?

Визуальные читатели заметят, что в примерах с `bIsFlag` мы фактически не экономим память. В логическом значении шаблу 8 byte. Но прямо, прямодушный вывод, используя 1 byte для определения параметров и 1 для битового флага? Так зачем же тогда нужны битовые флаги?

Они используются в двух случаях:

Случай №1. Если у вас много идентичных битовых флагов.

Вместо одной переменной `bIsFlag`, рассмотрим случай, когда у вас есть две переменные: `bIsFlag1` и `bIsFlag2`, каждая из которых может принимать значения. Если вы определите их как два отдельных логических значения, то вам потребуется 16 логических значений, а таким образом, 16 byte. Однако, с использованием битовых флагов вам потребуется только 10 byte (8 для определения параметров и 2 для каждой переменной `bIsFlag`). А вот если у вас будет 100 переменных `bIsFlag`, то, используя битовые флаги, вам потребуется 108 byte вместо 800. Чем больше идентичных переменных вам нужно, тем более значительной будет экономия памяти.

Давайте рассмотрим конкретный пример. Предположим, что вы создаете игру, в которой игрокам придется кормить с монстрами. Монстр, в свою очередь, может быть голодным или сытым, а также может пить или (выбранным случайным образом). В игре есть следующие типы еды: яд, молоко, сочно, вода, сорока, кислота, гарадин и слезка.

Чтобы отобразить, к какому типу этого монстр относится, мы можем использовать два логических значения на сопротивление (для одного монстра). Это 8 логических значений (рекомендуется для одного монстра = 8 byte).

Для 100 монстров это будет 800 переменных типа bool и 800 byte памяти.

А вот использова битовые флаги:

```
// Libs:
// Flags:
const unsigned char resistPoison = 0x01;
const unsigned char resistMilk = 0x02;
const unsigned char resistSoda = 0x04;
const unsigned char resistWater = 0x08;
const unsigned char resistGarad = 0x10;
const unsigned char resistSalt = 0x20;
const unsigned char resistAcid = 0x40;
const unsigned char resistTears = 0x80;

int main()
{
    // Определим переменную для хранения наших флагов
    const unsigned char isHungry = 0x01; // 00000001 (0-й бит)
    const unsigned char isThirsty = 0x02; // 00000010 (1-й бит)

    unsigned char monster = 0; // определим переменную от имени монстра
    monster |= isHungry | resistPoison | resistMilk | resistSoda | resistWater | resistGarad | resistSalt | resistAcid | resistTears; // монстр голодный и имеет сопротивление к всем видам еды
    monster |= isThirsty | resistPoison | resistMilk | resistSoda | resistWater | resistGarad | resistSalt | resistAcid | resistTears; // монстр пьет и имеет сопротивление к всем видам еды

    std::cout << "Monster is hungry?" << (bool(monster & isHungry)) << '\n';
    std::cout << "Monster is thirsty?" << (bool(monster & isThirsty)) << '\n';

    return 0;
}
```

Как нужно будет только 1 бит для хранения сопротивления каждого монстра и одинаковым типа в 8 byte для типов этак.

Таким образом, потребуется только 108 byte или примерно в 8 раз меньше памяти.

В большинстве программ, содержащий сотни или даже тысячи использованных битовых флагов не стоит добавляной сложности. Но в программах, где есть десятки тысяч или даже миллионы похожих объектов, за использованием может значительным сэкономить память. Согласитесь, знать о таком положении также не помешает!

Случай №2. Предположим, что у вас есть функция, которая может принимать любое комбинацию из 32 различных параметров. Одним из способов написания этой функции является использование 32 отдельных логических параметров:

```
void myFunction(bool bParam1, bool bParam2, bool bParam3, bool bParam4, bool bParam5, bool bParam6, bool bParam7, bool bParam8, bool bParam9, bool bParam10, bool bParam11, bool bParam12, bool bParam13, bool bParam14, bool bParam15, bool bParam16, bool bParam17, bool bParam18, bool bParam19, bool bParam20, bool bParam21, bool bParam22, bool bParam23, bool bParam24, bool bParam25, bool bParam26, bool bParam27, bool bParam28, bool bParam29, bool bParam30, bool bParam31, bool bParam32);
```

Затем, если вы захотите вызвать функцию с 10-м и 32-м параметрами, установленными как true — вам придется сделать следующее:

```
myFunction(false, false, false, false, false, false, false, false, true, false, false, false, false, false, false, false, false, false, false, false, false, false, false, false, false, false, false, false, false, true);
```

Т.к. перечислить все варианты как false, сорока 10-й 32 — оне true. Читать такой код сложно, да и требуется держать в памяти переданные номера значения параметров (10 и 32 или 11 и 33). Также пустовато не может быть эффективной.

А вот если определить функцию с помощью битовых флагов:

```
void myFunction(unsigned char mParams);
```

то можно выбрать и передавать только нужные параметры:

```
myFunction(mParam1 | mParam2);
```

Кроме того, что это необходимо, так также эффективнее и производительнее, так как включает только 2 операции (1 побитовое ИЛИ и 1 передача параметра).

Но почему в OpenGL используют битовые флаги вместо длинной последовательности логических значений.

Также, если у вас есть многочисленные битовые флаги и вам нужно поиск добавить параметры, вы можете просто определить битовый флаг. Нет необходимости изменять **прототип функции**, а это плюс к обеспечение обратной совместимости.

Введение в std::bitset

Все эти флаги, битовые флаги, операторы — всего слишком, на этот утомительно, на предар std C стиль, и Стандартной библиотеке C++ есть такой объект, как **std::bitset**, который упрощает работу с битовыми флагами.

Для его использования необходимо подключить заголовочный файл bitset, а затем определить переменную типа std::bitset, указав необходимое количество бит. Она должна быть кратной **типа sizeof-типа**.

```
#include <bitset>

std::bitset a, b(0x1c); // определим 8 бит
```

При значении std::bitset можно инициализировать начальным набором значений:

```
std::bitset a, b(0x1c[option]); // значение с основным целым
std::bitset a(0xff, 4); // значение с основным целым (4 бит)
```

Обратите внимание, наше начальное значение **конвертируется в двоичную систему**. Так как мы ввели шестнадцатеричное 2, то std::bitset преобразует его в двоичное 0000 0010.

В std::bitset есть 4 основных функции:

- **функция test()** — позволяет знать значение бита (0 или 1);
- **функция set()** — позволяет значить бита (если оне уже включено, то ничего не произойдет);
- **функция reset()** — позволяет выключить бита (если оне уже включено, то ничего не произойдет);
- **функция flip()** — позволяет изменить значение бит на противоположное (с 0 на 1 или с 1 на 0).

Каждая из этих функций принимает в качестве параметров позицию бит. Позиция крайнего правого бита (последнего) равна 0, затем последующий номер растет с каждым последующим битом вплоть (1, 2, 3, 4 и т.д.). Стоработь давать содержательные имена битовым индексам (либо путем присваивания их константным переменным, либо с помощью перечислений — о них мы поговорим на соответствующем уроке).

```
// Libs:
// Flags:
int main()
{
    // Определим переменную для хранения 8-битных значений
    // (от самого бита 0 до бита 7)
    const int mBits0 = 0;
    const int mBits1 = 1;
    const int mBits2 = 2;
    const int mBits3 = 3;
    const int mBits4 = 4;
    const int mBits5 = 5;
    const int mBits6 = 6;
    const int mBits7 = 7;

    int mBits;
    std::bitset<8> myBits(0x00); // определим 8-битную переменную
    mBits = 0x02; // присвоим 0x02 (00000010) к нашей переменной
    myBits |= (1 << mBits1); // включим второй бит (бит 1)
    myBits |= (1 << mBits2); // включим третий бит (бит 2)
    myBits &= ~(1 << mBits1); // выключим второй бит (бит 1)
    myBits ^= (1 << mBits2); // переключим третий бит (бит 2)

    std::cout << "Bit 0 has value: " << myBits.test(mBits0) << '\n';
    std::cout << "Bit 1 has value: " << myBits.test(mBits1) << '\n';
    std::cout << "All bits bits: " << myBits << '\n';

    return 0;
}
```

Результат выполнения программы:

```
Bit 0 has value: 0
Bit 1 has value: 0
All the bits: 00000000
```

Обратите внимание, отображение переменную `bits` в std::cout — выводятся значения всех бит в std::bitset.

Помимо, что инициализирование значения std::bitset рассматривается как двоичное, и то время как функции std::bitset используют позицию бит:

std::bitset также поддерживает стандартные побитовые операторы (`|`, `&` и `^`), которые также можно использовать (они полезны при проверке операций одновременно нескольких бит сразу с несколькими битами).

Вместо выполнения всех побитовых операций вручную, рекомендуется использовать std::bitset, так как он более удобен и менее подвержен ошибкам.

Битовые маски

Включение, выключение, переключение или запрашивание сразу нескольких бит можно осуществить в одной битовой операции. Когда мы соединяем отдельные биты вместе, в целях их модификации как группы, то это называется **битовой маской**.

Рассмотрим пример. В следующей программе мы просим пользователя ввести число. Затем, используя битовую маску, мы соединяем только последние 4 бита, значение которого и выводим в консоль.

```
// Libs:
// Flags:
int main()
{
    const unsigned int bmMask = 0xf; // Битовая маска для значения последних 4 бит (шестнадцатеричный литерал для 0000 0001)

    std::cout << "Enter an integer: ";
    int mInt;
    std::cin >> mInt;

    int mResult = mInt & bmMask; // сохраним только первые 4 значения

    std::cout << "The 4 low bits have value: " << mResult << '\n';

    return 0;
}
```

Результат выполнения программы:

```
Enter an integer: 151
The 4 low bits have value: 7
```

151 в десятичной системе равно 1001 0111 в двоичной. `bmMask` — это 0000 1111 в 8-битной двоичной системе. 1001 0111 & 0000 1111 = 0000 0111, что равно десятичному 7.

Пример с RGBA

Цветные дисплейные устройства, такие как телевизоры и мониторы, состоят из миллионов пикселей, каждый из которых может отображать точку цвета состоит из 3 точек: один красный, один зеленый и один синий (отсю **«RGB»** от англ. **red**, **green**, **blue**). Изменяя их интенсивность, можно воссоздать любой цвет. Количество цветов R, G и B в целом после представлено 8-битным целым или оттенкой. Например, цвет имеет R = 255, G = 0, B = 0, фиолетовый. R = 255, G = 0, B = 255, серый R = 127, G = 127, B = 127.

Используется еще и 4-значение, которое называется A, или от англ. **alpha**, которое отвечает за прозрачность. Если A = 0, то цвет полностью прозрачный. Если A = 255, то цвет непрозрачный.

В совокупности R, G, B и A составляют одно 32-битное целое число, с 8-битным для каждого компонента:

32-бит-ное значение RGBA			
31-24 биты	23-16 биты	15-8 биты	7-0 биты
RRRRRRRR	GGGGGGGG	BBBBBBBB	AAAAAAAA
red	green	blue	alpha

Следующая программа просит пользователя ввести 32-битное шестнадцатеричное значение, а затем извлекает 8-битные цветовые значения R, G, B и A:

```
// Libs:
// Flags:
int main()
{
    const unsigned int redMask = 0xff000000;
    const unsigned int greenMask = 0x00ff0000;
    const unsigned int blueMask = 0x0000ff00;
    const unsigned int alphaMask = 0x000000ff;

    std::cout << "Enter a 32-bit RGBA color value in hexadecimal (e.g. ff7f3300): ";
    int mInt;
    std::cin >> std::hex >> mInt; // ввод 32-битного значения в шестнадцатеричном виде

    // Извлекаем значения каждого цвета, используя маски
    // Нам нужно сдвинуть значения вправо, чтобы они были в диапазоне 0-255
    unsigned char red = (mInt & redMask) >> 24;
    unsigned char green = (mInt & greenMask) >> 16;
    unsigned char blue = (mInt & blueMask) >> 8;
    unsigned char alpha = (mInt & alphaMask);

    std::cout << "Your color components:" << '\n';
    std::cout << static_cast<int>(red) << " of 255 red\n";
    std::cout << static_cast<int>(green) << " of 255 green\n";
    std::cout << static_cast<int>(blue) << " of 255 blue\n";
    std::cout << static_cast<int>(alpha) << " of 255 alpha\n";

    return 0;
}
```

Результат выполнения программы:

```
Enter a 32-bit RGBA color value in hexadecimal (e.g. ff7f3300): ff7f3300
Your color components:
255 of 255 red
127 of 255 green
51 of 255 blue
0 of 255 alpha
```

В программе, приведенной выше, побитовое И используется для запроса 8-битного набора, который нас интересует, затем мы его сдвигаем вправо в диапазон от 0 до 255 для хранения и вывода.

Примечание: RGBA иногда может храниться как ARGB. В таком случае, главным байтом является альфа.

Заключение

Давайте кратко повторим то, как включить, выключить, переключить и запрашивать битовые флаги.

Для запроса битового состояния используется побитовое И:

```
flags & mask0;
```

Для включения бит используется побитовое ИЛИ:

```
flags |= mask0; // включим первый бит
flags |= mask1; // включим второй бит
```

Для выключения бит используется побитовое И с инвертированным литералом:

```
flags &= ~mask0; // выключим первый бит
flags &= ~mask1; // выключим второй бит
```

Для переключения между битовым состоянием используется побитовое исключающее ИЛИ (XOR):

```
flags ^= mask0;
flags ^= mask1;
```

Тест

Есть следующий фрагмент кода:

```
int main()
{
    unsigned char option_viewed = 0x01;
    unsigned char option_edited = 0x02;
    unsigned char option_liked = 0x04;
    unsigned char option_shared = 0x08;
    unsigned char option_deleted = 0x10;

    unsigned char myArticleFlags;

    return 0;
}
```

Примечание: Статьи — это myArticleFlags .

Задание №1

Добавьте строку кода, чтобы пометить статью как уже прочитанную (option_viewed).

Задание №2

Добавьте строку кода, чтобы проверить, были ли статьи удалены (option_deleted).

Задание №3

Добавьте строку кода, чтобы открыть статью от закрепленного места (option_favorited).

Задание №4

Почему следующие две строки идентичны?

```
myArticleFlags |= (option | option);
myArticleFlags |= option;
```

Ответы

Ответ №1

```
myArticleFlags |= option_viewed;
```

Ответ №2

```
if (myArticleFlags & option_deleted) ...;
```

Ответ №3

```
myArticleFlags &= ~option_favorited;
```

Ответ №4

Закона Де Моргана гласит, что если мы используем побитовое НЕ, то операторы И и ИЛИ меняются местами. Следовательно, `~(option | option)` становится `~option & ~option`.